

RK3566 上启用 OpenGL ES 2/3 与 Vulkan 1.1 的驱动路线深度研究报告

执行摘要

RK3566 集成的 3D GPU 在官方数据手册中被明确描述为“Embedded 3D GPU”，并声明可兼容 **OpenGL ES 1.1/2.0/3.2、OpenCL 2.0 与 Vulkan 1.1**。^① 这意味着从硬件能力角度，RK3566 的目标 API（GLES2/3 与 Vulkan 1.1）是“理论可达”的。

在**主线 Linux**（你目标的 6.1）上，RK356x 家族的 GPU 已以标准的 DeviceTree 节点形式进入上游 DTS：GPU 节点使用 `compatible = "rockchip,rk3568-mali", "arm,mali-bifrost"`，并给出了寄存器区、IRQ、时钟、devfreq OPP、功耗域等关键属性；板级 DTS 只需 `&gpu { mali-supply = <...>; status = "okay"; }` 即可启用。^② 因此从“内核侧启用 GPU”而言，主线 6.1 并不构成门槛（前提是你的板级供电/时钟/功耗域描述正确）。

在**用户态**，若选择开源路线（主线内核 `panfrost` + Mesa^③），可稳定获得 **OpenGL ES 3.1**（覆盖 GLES2/3 需求的大部分场景）。^④ 但对 RK3566 的 Mali-G52（Bifrost v7），开源 Vulkan（PanVK）目前在 Mesa 文档中对应 **Vulkan 1.0**，而非 Vulkan 1.1；并且 PanVK 仅在 Mali-G610 上声明达到 Vulkan 1.1/更高的正式一致性里程碑。^⑤

结论是：**若你“必须要 Vulkan 1.1（而不是 Vulkan 1.0）”，开源路线很可能不满足要求**；相对可行的是采用闭源 Mali 用户态库（libmali / rockchip-mali 等）+ 与之匹配的内核侧 Mali 驱动（通常为 `mali_kbase` 模块），并按 Vulkan Loader 的 ICD 机制安装 JSON 清单文件。^⑥

下文给出两条可落地的集成路径（开源/闭源），并补齐：RK3566 GPU IP 与 DTS/DTB 细节、闭源 blob 来源与下载链接、内核配置与设备树片段、用户态组件路径与 Vulkan ICD 配置、交叉编译与验证调试方法、以及常见踩坑。

RK3566 图形硬件与主线内核支持面

GPU IP 与官方能力声明

RK3566 数据手册在“Overview”段落直接写明：其嵌入式 3D GPU 使芯片可兼容 OpenGL ES 1.1/2.0/3.2、OpenCL 2.0 与 Vulkan 1.1。^① 这同时回答了“是否存在 Vulkan 1.1 的硬件前提”：**官方明确写了 Vulkan 1.1（至少在规格层面）**。^①

主线 DeviceTree 中的 GPU 节点形式

上游 `rk356x.dtsi` 已包含 GPU 节点定义（命名为 `gpu: gpu@fde60000`），其关键点包括：

- `compatible = "rockchip,rk3568-mali", "arm,mali-bifrost"`
- `reg = <0x0 0xfde60000 0x0 0x4000>`
- 三路 IRQ（job/mmu/gpu）与 `interrupt-names`
- 两个时钟输入：`clock-names = "gpu", "bus"`，并且包含一个 `scmi_clk` 来源和一个 `cru` 的 `CLK_GPU`

- `operating-points-v2 = <&gpu_opp_table>`、`power-domains = <&power RK3568_PD_GPU>`、`#cooling-cells` 等（用于 devfreq/热管理/功耗域）⁷

这对你非常关键：即使目标是 RK3566，主线 GPU 节点仍然使用 `rockchip,rk3568-mali` 这一 SoC-family 兼容串（上游就是这么做的），你无需发明新的 compatible。⁷

板级 DTS 的典型启用方式也已在上游示例中出现：例如 Quartz64-A 的板级 DTS 仅用 `&gpu { mali-supply = <&vdd_gpu>; status = "okay"; }` 就启用了 GPU。⁸ 另一份板级 dtsti（Radxa Zero 3 系列）同样用 `&gpu` 覆盖供电并置 `status = "okay"`。⁹

DT Binding 侧的约束与一致性

主线 DT binding（`arm,mali-bifrost.yaml`）对 Bifrost Mali GPU 的 binding 做了规范化，并在 compatible 列表中包含 `rockchip,rk3568-mali`。¹⁰ 这意味着你在主线内核上应尽量遵循该 binding 的属性集合（至少包括 reg、interrupts、clocks/clock-names、power-domains、operating-points-v2 等），否则驱动探测/时钟/功耗域/OPP 可能出现“探测到了但跑不起来”的典型故障。¹¹

方案对比与选型结论

对比表

选项名称	开源/闭源	可用 GLES 版本	Vulkan 支持	内核侧要求（主线 6.1 视角）	集成难度	稳定性/维护性
主线开源： <code>panfrost</code> (kernel) + Mesa Panfrost(GLES)	开源	GLES 3.1（覆盖 GLES2/3 需求多数） ⁴	无或不启用 Vulkan（只跑 GLES） ⁴	需要正确 DT（ <code>rockchip,rk3568-mali</code> / 供电/时钟/功耗域/OPP），并启用 Rockchip DRM/KMS + <code>panfrost</code> ²	中	高（主线友好；更新随内核/Mesa） ⁴
主线开源： <code>panfrost</code> + Mesa PanVK (Vulkan)	开源	GLES 3.1 ⁴	Vulkan 1.0 (Mali-G52 / Bifrost v7) ⁴	同上；另需 Mesa 构建启用 <code>-Dvulkan-drivers=panfrost</code> ⁴	中-高	中（PanVK 在 G52 上仍属非一致性状态） ⁴
闭源：Rockchip/厂商 <code>libmali</code> （含 Vulkan）+ <code>mali_kbase</code> 内核模块	闭源（用户态 blob + 内核模块）	目标为 GLES2/3（具体上限取决于 blob 版本；官方规格写到 GLES 3.2） ¹	目标 Vulkan 1.1（官方规格写到 Vulkan 1.1） ¹	需要能在 6.1 上构建并加载 <code>mali_kbase</code> （通常 DKMS/外置模块）；GPU DT 仍需正确供电/时钟/功耗域 ¹²	高	中-低（版本耦合强；主线升级成本高） ¹³

选项名称	开源/ 闭源	可用 GLES 版本	Vulkan 支持	内核侧要求（主线 6.1 视角）	集 成 难 度	稳定性/维 护性
“只装闭源用户态、 继续用 panfrost 内 核”混搭	混合 （不 推 荐）	不稳定/不 匹配	不稳定/ 不匹配	用户态与内核 uAPI 不一 致风险极高（应按同一 栈配套） ¹⁴	很高	低

选型建议

如果你的目标是“在主线 6.1 上稳定获得 GLES2/3”，并且 Vulkan 只要“有即可”（可接受 Vulkan 1.0 或仅 GLES），优先建议走 开源 **panfrost** + Mesa。¹⁵

如果你明确要求“Vulkan 1.1（而不是 1.0）”，结合 Mesa 文档对 Mali-G52 的 Vulkan 版本表述，开源 PanVK 在该 GPU 上目前仅标到 Vulkan 1.0，因此更现实的路径是走 闭源 **Mali 栈 (libmali/rockchip-mali) + mali_kbase**，并按 Vulkan Loader 的 ICD JSON 方式安装。¹⁶

开源路径：主线 panfrost + Mesa（推荐的“主线友好”方案）

组件交互与调用链

```

flowchart LR
    subgraph App[应用层]
        A1[OpenGL ES 应用]
        A2[Vulkan 应用]
    end

    subgraph User[用户态图形栈]
        MESA[Mesa: libEGL/libGLES + DRI 驱动]
        PANVK[PanVK ICD\n(Mesa Vulkan driver)]
        VULKANLOADER[Libvulkan loader]
    end

    subgraph Kernel[内核态]
        DRM[Rockchip DRM/KMS\n(VOP2/HDMI 等)]
        PF[panfrost DRM 驱动\n/renderD* 节点]
    end

    A1 --> MESA --> PF --> DRM
    A2 --> VULKANLOADER --> PANVK --> PF --> DRM

```

该图对应 Mesa 文档中描述的 Panfrost/PanVK 栈：Panfrost 提供 GLES，PanVK 提供 Vulkan；两者都依赖内核 **panfrost** 驱动与 DRM/KMS 显示子系统。¹⁷

能力边界：GL ES 与 Vulkan 版本

对 RK3566 的 Mali-G52 (Bifrost v7)，Mesa 文档表格给出的结论是：

- OpenGL ES: **3.1** (意味着 GLES2/3 都可覆盖, 但不等同于 GLES 3.2) ⁴
- Vulkan (PanVK) : **1.0** (不是 1.1) ⁴

因此：如果你“必须 Vulkan 1.1”，请直接跳到闭源路径。 ¹⁸

内核侧要求 (DRM/KMS、模块、DeviceTree)

设备树最小启用片段 (板级 DTS 覆盖)

来自上游板级 DTS 的典型写法如下 (你可以视为最小“启用 GPU”补丁形态)： ¹⁹

```
/* 在你的板级 .dts 中 */
&gpu {
    mali-supply = <vdd_gpu>; /* 也可能是 vdd_gpu_npu, 取决于板级供电设计 */
    status = "okay";
};
```

你还应确保 SoC 层的 GPU 节点 (通常来自 `rk356x.dtsi` / `rk3566*.dtsi`) 完整存在并符合上游定义, 包括 interrupts、clocks、power-domains、operating-points-v2 等 (否则容易出现“probe 成功但 GPU 一直 0%/hang”的现象)。 ²⁰

SoC 层 GPU 节点参考 (用于核对你的 DT 是否缺项)

下面是上游 `rk356x.dtsi` 中 GPU 节点的关键字段 (建议你对照核查自己的 DTB 是否等价)： ⁷

```
gpu: gpu@fde60000 {
    compatible = "rockchip,rk3568-mali", "arm,mali-bifrost";
    reg = <0x0 0xfde60000 0x0 0x4000>;
    interrupts = <GIC_SPI 40 IRQ_TYPE_LEVEL_HIGH>,
        <GIC_SPI 41 IRQ_TYPE_LEVEL_HIGH>,
        <GIC_SPI 39 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "job", "mmu", "gpu";
    clocks = <&scmi_clk 1>, <&cru CLK_GPU>;
    clock-names = "gpu", "bus";
    #cooling-cells = <2>;
    operating-points-v2 = <&gpu_opp_table>;
    power-domains = <&power RK3568_PD_GPU>;
    status = "disabled";
};
```

内核配置建议（以 6.1 为目标）

由于板卡/发行版未指定，下列以“功能域”为导向列出你通常需要开启的配置（名称可能随发行版 `defconfig` 有差异，但功能意图不变）：

- DRM/KMS：启用 Rockchip DRM（用于 VOP2/HDMI 等扫描输出）²¹
- GPU：启用 `panfrost` DRM 驱动（Mali Midgard/Bifrost）²²
- IOMMU/电源管理：RK356x GPU 节点本身声明了 `power-domains` 与 OPP 表，意味着你需要相应的功耗域与 `devfreq` 体系正常工作²³

备注：在 RK356x 的 GPU 上游合入讨论中，有测试者特别指出某些 EVB 上 GPU 功耗域供电需要在 DT 中设为 `always-on` 才能稳定运行；这类板级差异往往来自 PMIC 供电拓扑与上电时序。

²⁴

用户态组件（libEGL/libGLES、Vulkan loader、ICD）

使用系统包（最省事的方式）

若你的发行版是 Debian²⁵ / Ubuntu²⁶ 系列（或衍生），一般做法是安装 Mesa（OpenGL ES）与 Vulkan 工具/loader：

```
# 典型包名需按发行版调整（这里给出常见组合）
sudo apt update
sudo apt install -y mesa-utils mesa-utils-extra \
libegl1 libgles2 \
vulkan-loader vulkan-tools mesa-vulkan-drivers
```

要点是：Vulkan loader 会在标准目录查找 ICD JSON（例如 `/usr/share/vulkan/icd.d`、`/etc/vulkan/icd.d` 等）。²⁷

`vulkaninfo` 属于 Vulkan-Tools 项目，用于打印 GPU/Vulkan 能力，是验证 Vulkan 路线的核心工具之一。

²⁸

自行编译 Mesa（当你需要更高版本的 PanVK/修复）

Mesa 文档给出了对 Panfrost/PanVK 的构建参数示例：`-Dvulkan-drivers=panfrost -Dgallium-drivers=panfrost`；并强调 Panfrost 编译器在构建时需要 LLVM，且跨编译时可将相关工具在 host 端预构建后再用于 target 侧。⁴

```
# 仅示意 —— 你需准备 meson 交叉文件与 sysroot
meson setup build \
-Dgallium-drivers=panfrost \
-Dvulkan-drivers=panfrost \
-Dplatforms=wayland,x11,drm
ninja -C build
ninja -C build install
```

开源路径的精简集成步骤（可执行清单）

1. 在主线 6.1 内核中启用 Rockchip DRM/KMS 与 `panfrost`，并确保你的 DT 使用上游兼容串（`rockchip,rk3568-mali` + `arm,mali-bifrost`）与正确的 clocks/power-domains/OPP。²⁰
2. 在板级 DTS 将 GPU `status` 置为 `okay`，并绑定正确的 `mali-supply`（参考 Quartz64/Radxa 的用法）。¹⁹
3. 启动后确认内核 GPU 节点已 probe（通常会出现 DRM render 节点），再安装/升级 Mesa（必要时自行编译）。¹⁷
4. 用 `eglinfo` / `glmark2-es2-wayland` / `vulkaninfo` 验证（见“验证调试”章节）。²⁹

闭源路径：Mali 用户态 blob + `mali_kbase`（面向 Vulkan 1.1 的现实路线）

闭源组件的“已知可获取”来源与下载入口

Rockchip 官方 Debian 仓库中的 `rockchip-mali` 包

Rockchip³⁰ 的公开仓库目录中提供了 `rockchip-mali_1.9-12_arm64.deb`（文件约 56MB）。³¹ 这类包通常用于在 Rockchip 系列平台上分发用户态 Mali 库（EGL/GLES/可能包含 Vulkan 变体），但其内部具体包含哪些 GPU 型号与哪些后端（x11/wayland/gbm/vulkan）在本报告中无法从目录页直接枚举，因此对“精确文件名列表/so 名称/是否含 G52 Vulkan 变体”在此标记为：**未指定（unspecified）**。³¹

你仍可按以下方式直接下载并查看包内容（在你的构建机环境中）：

```
wget https://repo.rock-chips.com/edge/debian-release-v2.0.0/pool/main/r/rockchip-mali/rockchip-mali_1.9-12_arm64.deb
dpkg-deb -c rockchip-mali_1.9-12_arm64.deb | less
dpkg-deb -I rockchip-mali_1.9-12_arm64.deb | sed -n '1,120p'
```

第三方/生态仓库（用于参考包名与变体命名规则）

Radxa³² 的文档在“验证用户态驱动”处给出了一个对 rk356x 的实例：`libmali-bifrost-g52-g13p0-x11-wayland-gbm`，版本号 `1.9-1`，来自其 `rk3566-bookworm-test` 仓库。³³ 这为你提供了两个关键线索：

- RK3566 的闭源用户态通常以 **Bifrost/G52/g13p0** 这类命名，且会区分 X11/Wayland/GBM 平台后端。³⁴
- 在一些发行版镜像中，“默认”可能是开源 Panfrost，而闭源 Mali 需要额外安装并切换。³⁵

在 OpenEmbedded³⁶ 的 Layer Index 中，`rockchip-libmali` 被标注为 License = **CLOSED**（闭源），这从工程集成角度佐证了其 blob 属性。³⁷

内核侧：为何必须引入 `mali_kbase`，以及它与主线的关系

闭源 Mali 用户态通常不走主线 DRM `panfrost` 的 uAPI，而是依赖 ARM Mali 的 kernel driver (`kbase`) 在系统中提供 `/dev/malixx` 设备文件，完成用户态到内核态的 `ioctl` 交互。相关代码注释中明确提到通过 `/dev/malixx` 提供 userspace access。³⁸

LinuxFactory 的 Bifrost 指南进一步说明：Bifrost 驱动以内核模块形式提供，模块名为 `mali_kbase.ko`，并通过包（示例为 `mali-bifrost-wayland-driver`）安装；重启后可用 `lsmod | grep mali_kbase` 验证模块加载。¹³ 这与“闭源用户态 + kbase 模块”的典型栈结构一致：**用户态 blob 与内核模块强耦合**。³⁹

因此，若你坚持“主线 Linux 6.1 + 闭源 GLES/Vulkan”，你的实际任务不是“把 blob 塞进主线”，而是：

- 主线 6.1 保持不变（或少量补丁），
- 额外构建并加载与 blob 匹配的 `mali_kbase` 外置模块（DKMS 或手工 out-of-tree），
- 并确保 DT 仍满足 GPU 的 clocks/power-domains/mali-supply/OPP。⁴⁰

Vulkan 侧：ICD JSON 的放置位置与选择机制（必须掌握）

Vulkan loader 会在 Linux 上的多个目录查找 ICD manifest（JSON），包括：`/usr/share/vulkan/icd.d/`、`/etc/vulkan/icd.d/`、`/usr/local/share/vulkan/icd.d/`、以及用户目录下的 `$HOME/.local/share/vulkan/icd.d` 等。²⁷

同时，它支持通过环境变量 `VK_ICD_FILENAMES` 强制指定一个或多个 ICD JSON 的完整路径列表（用冒号分隔）。⁴¹

这意味着闭源 Vulkan 的“可用性”高度依赖两件事：

- 1) 你安装的 blob 是否真的包含 Vulkan driver `.so`（未指定，需要你解包确认）。³¹
- 2) 你的 ICD JSON 是否放在 loader 能找到的位置（或用 `VK_ICD_FILENAMES` 强制指定）。⁴²

社区示例（Mali-G610，非 RK3566）展示了如何从 Rockchip 的 `rockchip-mali` 包中选择一个带 `-vulkan` 后缀的库文件，并在 `/etc/vulkan/icd.d/` 下创建 `mali.json`。⁴³ 你在 RK3566 上可以复用同一“方法论”，只是库文件名应替换为 **G52/Bifrost 对应的 Vulkan 库**（具体名称：**未指定，需要你从包内实际文件名确认**）。⁴⁴

闭源路径的精简集成步骤（面向主线 6.1，强调可操作性）

重要前提：你需要自行确认你拿到的 `libmali/Vulkan .so` 与 `mali_kbase` 的版本是否匹配；不匹配时常见现象是初始化失败、`ioctl` 返回不兼容错误、或应用启动即崩溃（具体错误与版本相关，本报告不做臆测）。³⁹

1. **准备主线 6.1 内核与 DT**：确保 GPU 节点与供电（`mali-supply`）、时钟、功耗域、OPP 表和上游一致（参考 `rk356x.dtsi` 定义与板级 `&gpu` 覆盖）。²
2. **安装/部署用户态 blob**：优先使用 Rockchip 官方 `rockchip-mali_1.9-12_arm64.deb`（下载链接见上），并用 `dpkg-deb -c` 解包确认是否存在 G52 对应的 GLES/EGL/Vulkan 变体库。⁴⁵
3. **构建并加载 `mali_kbase` 模块**：选择 DKMS（更利于多内核版本维护）或手工 out-of-tree 模块方式；加载后用 `lsmod | grep mali_kbase` 验证。¹³
4. **确认设备节点**：确认 `/dev/mali0`（或更一般的 `/dev/malixx`）已出现且权限正确。³⁸
5. **建立 Vulkan ICD JSON**：将 ICD JSON 放入 `/etc/vulkan/icd.d/` 或 `/usr/share/vulkan/icd.d/`，或使用 `VK_ICD_FILENAMES` 指定。⁴²
6. **验证 Vulkan/GLES**：使用 `vulkaninfo`、GLES 测试与系统日志联合定位问题（见下一章）。⁴⁶

闭源 Vulkan ICD JSON 示例（你必须将 `library_path` 改成你系统里实际存在的 G52 Vulkan `.so` 路径；此处仅展示格式）：

```
{
  "file_format_version": "1.0.0",
```

```
"ICD": {
  "library_path": "/usr/lib/aarch64-linux-gnu/<YOUR_G52_VULKAN_SO>",
  "api_version": "1.1.0"
}
}
```

其中 `api_version` 的设定应以你 blob 对 Vulkan 的实际支持为准；RK3566 数据手册写到 Vulkan 1.1，但 blob 是否按 Vulkan 1.1 对外声明、以及 ICD JSON 应填什么版本，仍需你以实际 `vulkaninfo` 输出为准（否则会被 loader 视为不兼容或限制测试覆盖面）。⁴⁷

集成、交叉编译与验证调试

集成步骤时间线（开源 vs 闭源）

```
flowchart TD
    S[开始：确定目标内核=6.1 与目标API] --> Q{是否硬性要求 Vulkan 1.1?}
    Q -- 否/可接受 Vulkan 1.0 或仅 GLES --> OSS[走开源：panfrost + Mesa]
    Q -- 是 --> BLOB[走闭源：libmali/rockchip-mali + mali_kbase + Vulkan ICD]

    OSS --> OSS_K[内核：启用 Rockchip DRM + panfrost\nNT: &gpu status=okay + mali-supply]
    OSS_K --> OSS_U[用户态：安装/编译 Mesa\n(启用 panfrost + 可选 PanVK)]
    OSS_U --> OSS_V[验证：eglinfo/glmark2-es2-wayland/vulkaninfo]

    BLOB --> B_K[内核：构建并加载 mali_kbase\nNT: 同样保证 clocks/power/opp/supply]
    B_K --> B_U[用户态：安装 blob (rockchip-mali 或 libmali-*)\n挑选正确后端(x11/wayland/gbm)]
    B_U --> B_ICD[配置 Vulkan ICD JSON\n(/etc/vulkan/icd.d 或 VK_ICD_FILENAMES)]
    B_ICD --> B_V[验证：/dev/malixx + vulkaninfo + GLES 测试]

    OSS_V --> E[收尾：性能/稳定性调优]
    B_V --> E
```

该流程图中的关键分叉依据来自 Mesa 文档对 Mali-G52 的 Vulkan 版本表述（开源 Vulkan 1.0）与 RK3566 数据手册的 Vulkan 1.1 规格描述。¹⁸

交叉编译与工具链要点

内核（aarch64）

典型交叉编译方式如下（以 GNU toolchain 为例）：

```
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-

make <your_defconfig>
make -j$(nproc) Image modules dtbs
```

你需要确保最终 DTB 中包含上游 GPU 节点（`rockchip,rk3568-mali` + `arm,mali-bifrost`）及板级 `&gpu` 覆盖供电。²

Mesa (Panfrost/PanVK)

Mesa 文档明确指出：Panfrost 编译器在构建时需要 LLVM；在跨编译环境里可先在 host 构建必要工具再用于 target 构建。⁴

因此当你在 Buildroot/Yocto/自建 sysroot 中编 Mesa 时，要重点处理 LLVM 与 meson 的 host-tools/target-libs 分离（否则常见错误是构建阶段缺失编译器工具）。

验证与调试 (GLES / Vulkan / 内核日志)

开源路径验证 (panfrost)

- 1) 确认 DT 生效与驱动加载（示例板级 DTS 已启用 &gpu）。⁴⁸
- 2) 运行 GLES 基准：Rockchip 文档页与 LinuxFactory 指南均推荐 glmark2-es2-wayland（尤其在 Wayland 环境下）。⁴⁹
- 3) Vulkan（若启用 PanVK）：运行 vulkaninfo 并观察 apiVersion 与设备枚举。vulkaninfo 的用途与来源见 Vulkan-Tools 项目说明。²⁸

```
# GLES
glmark2-es2-wayland

# Vulkan
vulkaninfo | less
```

闭源路径验证 (libmali + mali_kbase)

- LinuxFactory 指出 mali_kbase 模块应被加载，可用 lsmod | grep mali_kbase 检查。¹³
- 设备节点通常为 /dev/malixx（例如 /dev/mali0），用于 userspace 与内核模块通信。³⁸
- Vulkan ICD JSON 放置路径遵循 Khronos（Vulkan loader）定义的查找目录；如需强制选择，使用 VK_ICD_FILENAMES。⁴²

```
# 1) 检查模块与设备节点
lsmod | grep mali_kbase
ls -l /dev/mali*

# 2) Vulkan loader 级别排错（强烈推荐）
VK_LOADER_DEBUG=all vulkaninfo 2>&1 | tee /tmp/vk_loader.log

# 3) 强制指定 ICD（当系统存在多 ICD 或路径不一致时）
export VK_ICD_FILENAMES=/etc/vulkan/icd.d/mali.json
vulkaninfo | head -n 80
```

GPU 频率/负载与基本 sysfs 观察

Rockchip 的 Mali 文档页提供了 devfreq 相关的 sysfs 读取/设置示例（包括 load、available_frequencies、切到 performance governor、以及用 strings libMali.so | grep rk_so_ver 查看版本字段）。⁵⁰

```
cat /sys/class/devfreq/*.gpu/load
cat /sys/class/devfreq/*.gpu/available_frequencies
echo performance | sudo tee /sys/class/devfreq/*.gpu/governor
strings /usr/lib/aarch64-linux-gnu/libmali.so | grep rk_so_ver || true
```

常见坑点与规避思路

- **DT 只 “status=okay” 但供电/功耗域没对上**：RK356x GPU 节点依赖 `mali-supply`、`power-domains` 与 `operating-points-v2`；缺项或 regulator 未上电会导致 GPU 无法稳定进工作状态。⁵¹
- **Vulkan “找不到驱动/ERROR_INCOMPATIBLE_DRIVER”**：优先检查 ICD JSON 是否放在 Vulkan loader 的标准目录（`/usr/share/vulkan/icd.d`、`/etc/vulkan/icd.d` 等），或用 `VK_ICD_FILENAMES` 强制指定路径，并开启 `VK_LOADER_DEBUG` 获取详细查找日志。⁴²
- **误以为开源即可 Vulkan 1.1**：Mesa 文档对 Mali-G52（Bifrost v7）明确列为 Vulkan 1.0；若你的项目硬需求是 Vulkan 1.1，请避免在开源路径上耗费过多时间。¹⁸
- **闭源栈版本耦合**：闭源用户态 blob 与 `mali_kbase` 模块强耦合，LinuxFactory 的说明已经体现其以独立包/模块形式分发并需要重启/重装 DKMS 来适配新内核。¹³

参考来源与关键补丁线索

- Rockchip³⁰ RK3566 数据手册（Rev 1.1），明确 Vulkan 1.1 / GLES 3.2 规格：¹
- 主线 Linux kernel⁵² DTS： `rk356x.dtsi` 的 GPU 节点（`rockchip,rk3568-mali` + `arm,mali-bifrost`，含 clocks/IRQs/power-domains/OPP）：⁷
- 上游板级 DTS 启用示例（`&gpu { mali-supply = ...; status="okay"; }`）：¹⁹
- DT bindings（Mali Bifrost，含 RK3568 compatible）：¹⁰
- Mesa³ Panfrost/PanVK 文档（Mali-G52：GLES 3.1，Vulkan 1.0；构建参数与 LLVM 要求）：⁴
- DRM Panfrost 内核文档（内核侧驱动说明）：⁵³
- Khronos Group⁵⁴ Vulkan loader（ICD JSON 查找目录、`VK_ICD_FILENAMES`）：⁴²
- Vulkan loader/工具的官方仓库与说明：⁵⁵
- Rockchip 公共仓库中的 `rockchip-mali`（闭源用户态包下载入口）：³¹
- RK356x GPU 上游合入相关讨论与注意事项（含“某些板卡需 always-on 供电”的提示）：²⁴
- 社区示例：从 Rockchip `rockchip-mali` 包中手工创建 Vulkan ICD JSON（示例基于 Mali-G610，但 ICD 方法对 RK3566 同样适用）：⁴³

¹ ¹⁶ ⁴⁷ <https://dl.xkwy2018.com/downloads/RK3568/Rockchip-RK3566-Datasheet-V1.1-20210305.pdf>
<https://dl.xkwy2018.com/downloads/RK3568/Rockchip-RK3566-Datasheet-V1.1-20210305.pdf>

² ⁷ ²⁰ ²³ ³² ⁴⁰ ⁵¹ ⁵⁴ <https://kernel.googlesource.com/pub/scm/linux/kernel/git/thierry.reding/linux-pwm/%2B/for-next/arch/arm64/boot/dts/rockchip/rk356x.dtsi>
<https://kernel.googlesource.com/pub/scm/linux/kernel/git/thierry.reding/linux-pwm/%2B/for-next/arch/arm64/boot/dts/rockchip/rk356x.dtsi>

³ ⁴ ⁵ ¹⁵ ¹⁷ ¹⁸ ³⁰ <https://docs.mesa3d.org/drivers/panfrost.html>
<https://docs.mesa3d.org/drivers/panfrost.html>

⁶ ¹² ¹³ ¹⁴ ²⁹ ³⁶ ³⁹ ⁵² https://docs.linuxfactory.or.kr/guides/gpu_bifrost.html
https://docs.linuxfactory.or.kr/guides/gpu_bifrost.html

8 19 48 <https://kernel.googlesource.com/pub/scm/linux/kernel/git/torvalds/linux/%2B/858fd168a95c5b9669aac8db6c14a9aeab446375/arch/arm64/boot/dts/rockchip/rk3566-quartz64-a.dts>
<https://kernel.googlesource.com/pub/scm/linux/kernel/git/torvalds/linux/%2B/858fd168a95c5b9669aac8db6c14a9aeab446375/arch/arm64/boot/dts/rockchip/rk3566-quartz64-a.dts>

9 <https://kernel.googlesource.com/pub/scm/linux/kernel/git/mmind/linux-rockchip/%2B/for-next/arch/arm64/boot/dts/rockchip/rk3566-radxa-zero-3.dtsi>
<https://kernel.googlesource.com/pub/scm/linux/kernel/git/mmind/linux-rockchip/%2B/for-next/arch/arm64/boot/dts/rockchip/rk3566-radxa-zero-3.dtsi>

10 11 **arm,mali-bifrost.yaml**
https://www.kernel.org/doc/Documentation/devicetree/bindings/gpu/arm%2Cmali-bifrost.yaml?utm_source=chatgpt.com

21 <https://lwn.net/Articles/892340/>
<https://lwn.net/Articles/892340/>

22 53 **drm/Panfrost Mali Driver — The Linux Kernel documentation**
https://dri.freedesktop.org/docs/drm/gpu/panfrost.html?utm_source=chatgpt.com

24 <https://patchew.org/linux/20220209215549.94524-1-michael.riesch%40wolfvision.net/>
<https://patchew.org/linux/20220209215549.94524-1-michael.riesch%40wolfvision.net/>

25 26 27 41 42 https://www.khronos.org/assets/uploads/developers/library/2017-vulkan-loader-webinar/VulkanLoaderDeepDive_Khronos_Mar17.pdf
https://www.khronos.org/assets/uploads/developers/library/2017-vulkan-loader-webinar/VulkanLoaderDeepDive_Khronos_Mar17.pdf

28 46 <https://github.com/KhronosGroup/Vulkan-Tools/blob/master/vulkaninfo/vulkaninfo.md>
<https://github.com/KhronosGroup/Vulkan-Tools/blob/master/vulkaninfo/vulkaninfo.md>

31 44 45 <https://repo.rock-chips.com/edge/debian-release-v2.0.0/pool/main/r/rockchip-mali/>
<https://repo.rock-chips.com/edge/debian-release-v2.0.0/pool/main/r/rockchip-mali/>

33 34 <https://docs.radxa.com/en/som/cm/cm5/radxa-os/mali-gpu>
<https://docs.radxa.com/en/som/cm/cm5/radxa-os/mali-gpu>

35 <https://docs.radxa.com/en/rock5/rock5b/radxa-os/mali-gpu>
<https://docs.radxa.com/en/rock5/rock5b/radxa-os/mali-gpu>

37 <https://layers.openembedded.org/layerindex/recipe/347513/>
<https://layers.openembedded.org/layerindex/recipe/347513/>

38 https://chromium.googlesource.com/chromiumos/third_party/kernel/%2B/chromeos-4.19/drivers/gpu/arm/bifrost/mali_kbase_defs.h
https://chromium.googlesource.com/chromiumos/third_party/kernel/%2B/chromeos-4.19/drivers/gpu/arm/bifrost/mali_kbase_defs.h

43 <https://github.com/Bleach665/Mali610Vulkan>
<https://github.com/Bleach665/Mali610Vulkan>

49 50 https://opensource.rock-chips.com/wiki_Mali
https://opensource.rock-chips.com/wiki_Mali

55 <https://github.com/KhronosGroup/Vulkan-Loader>
<https://github.com/KhronosGroup/Vulkan-Loader>